Case Study: Student Marks and Result Analysis

Problem Statement:

Student Marks and Result Analysis: Calculate the total marks and percentage of a student across several subjects, apply grace marks for students scoring between 30 and 34, identify failed subjects, and determine the final result and class based on the percentage.

Objective

- Accept the number of subjects and marks obtained in each subject.
- Check whether the entered marks are valid.
- Apply grace marks if the marks are between 30 and 34.
- Calculate total marks and percentage.
- Identify failed subjects and determine the final result.
- Display the student's class based on the percentage.

Assumptions

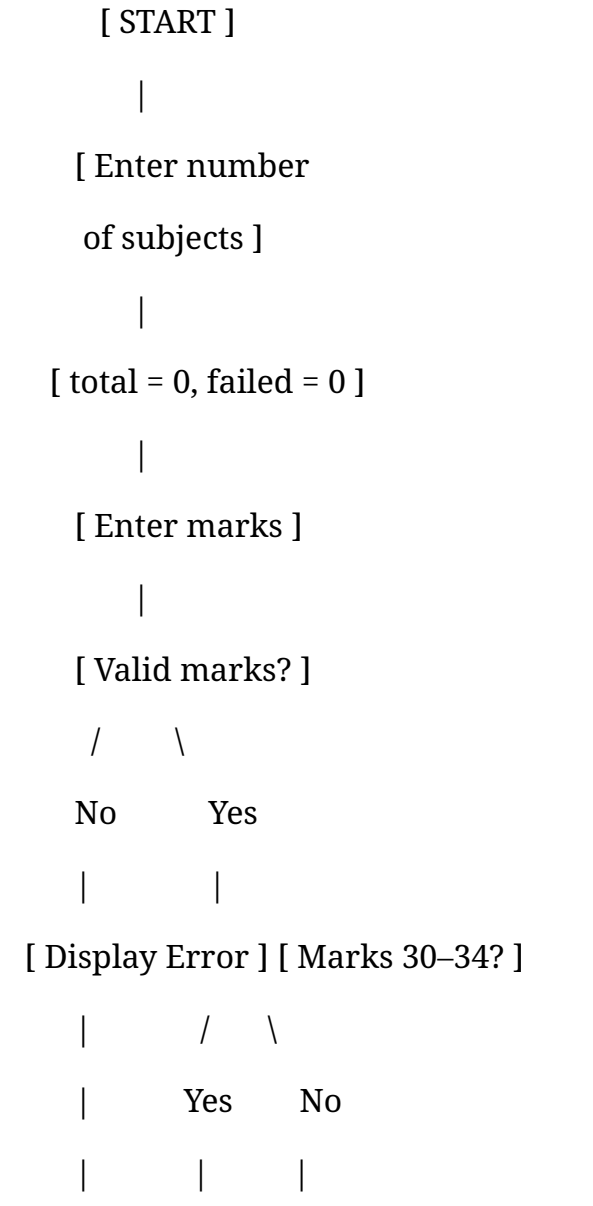
- Marks are entered out of 100 for each subject.
- The pass mark is 35.
- Students scoring between 30 and 34 receive grace marks to reach 35.
- Invalid marks must be less than 0 or greater than 100.
- Class is determined based on the final percentage.

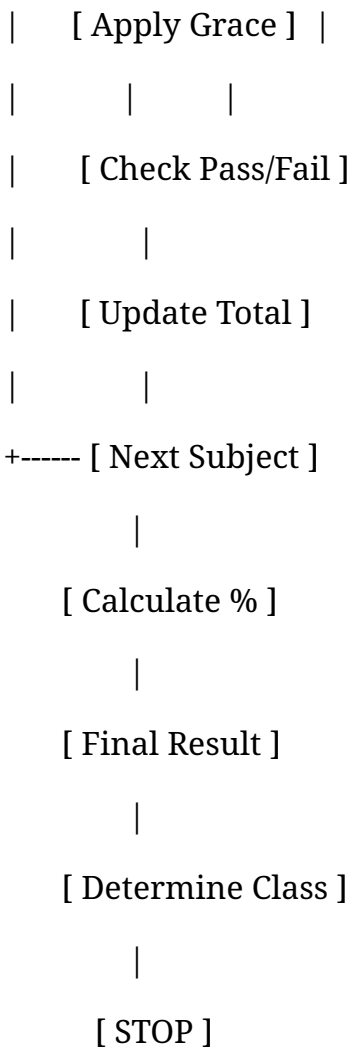
Algorithm

1. Start.
2. Read the number of subjects n.
3. Initialize total = 0 and failed = 0.
4. Repeat for each subject from 1 to n.
5. Read the marks obtained.
6. If marks are invalid, display an error message and continue to the next subject.

7. If marks are between 30 and 34, apply grace marks and set marks to 35.
8. Display the final marks.
9. If marks are below 35, increase the failed count; otherwise, display Pass.
10. Add the final marks to the total.
11. Calculate total maximum marks and percentage.
12. If any subject is failed, display Fail; otherwise, display Pass.
13. Determine the class based on the percentage.
14. Stop.

Flowchart





Pseudocode

START

Read n

total = 0

failed = 0

FOR each subject from 1 to n

Read marks

IF marks < 0 OR marks > 100

```
        Display "Invalid marks"
        CONTINUE
    ENDIF

    IF marks >= 30 AND marks < 35
        marks = 35
        Display "Grace marks are applied"
    ENDIF

    Display final marks

    IF marks < 35
        Display "Fail"
        failed = failed + 1
    ELSE
        Display "Pass"
    ENDIF

    total = total + marks
ENDFOR

Calculate percentage
Display total and percentage

IF failed > 0
    Display "Final Result: Fail"
```

ELSE

Display "Final Result: Pass"

ENDIF

Determine class using percentage

STOP

Python Source Code

```
n = int(input("Enter the number of subjects: "))

total = 0
failed = 0

for i in range(1, n + 1):
    marks = int(input("Enter the marks: "))

    if marks < 0 or marks > 100:
        print("Invalid marks!")
        continue

    if marks >= 30 and marks < 35:
        marks = 35
        print("Grace marks are applied!")

    print("Final marks:", marks)
```

```
    if marks < 35:
        print("Failed")
        failed = failed + 1
    else:
        print("Pass")

    total = total + marks

print("\nTotal marks:", total, "/", n * 100)

percentage = (total / (n * 100)) * 100
print("Percentage:", round(percentage, 2), "%")

if failed > 0:
    print("Final result: Fail")
else:
    print("Final result: Pass")

if percentage >= 75:
    print("Distinction")
elif percentage >= 60:
    print("First class")
elif percentage >= 50:
    print("Second class")
else:
```

```
    print("Third class")
```

Note: I corrected the invalid-marks condition from marks < 0 and marks > 100 to marks < 0 or marks > 100. A number cannot be both less than 0 and greater than 100 at the same time.

Test Cases

Test Case 1: All subjects passed

Input:

Enter the number of subjects: 3

Enter the marks: 80

Enter the marks: 75

Enter the marks: 90

Output:

Final marks: 80

Pass

Final marks: 75

Pass

Final marks: 90

Pass

Total marks: 245 / 300

Percentage: 81.67 %

Final result: Pass

Distinction

Test Case 2: Grace marks applied

Input:

Enter the number of subjects: 2

Enter the marks: 32

Enter the marks: 70

Output:

Grace marks are applied!

Final marks: 35

Pass

Final marks: 70

Pass

Total marks: 105 / 200

Percentage: 52.5 %

Final result: Pass

Second class

Conclusion

This program calculates student marks, total percentage, and final results. It uses loops, conditional statements, arithmetic operators, and user input in Python. Grace marks are applied to eligible students, and the program identifies failed subjects and determines the student's class based on the percentage.

Case Study: Bank Transaction and Balance Management System

Problem Statement:

Bank Transaction and Balance Management System: Process multiple bank transactions, including deposits and withdrawals, while checking withdrawal limits and available balance. Calculate total deposits, total withdrawals, and the final account balance.

Objective

- Initialize the account balance and withdrawal limit.
- Accept the number of transactions.
- Allow the user to deposit or withdraw money.
- Validate transaction amounts and choices.
- Check withdrawal limits and available balance.
- Calculate total deposits, total withdrawals, and final balance.

Assumptions

- The initial account balance is ₹1,000.
- The maximum withdrawal per transaction is ₹500.
- The user enters the transaction type and amount.
- Negative transaction amounts are invalid.
- Deposits increase the balance, and successful withdrawals decrease it.
- Invalid transactions do not change the balance.

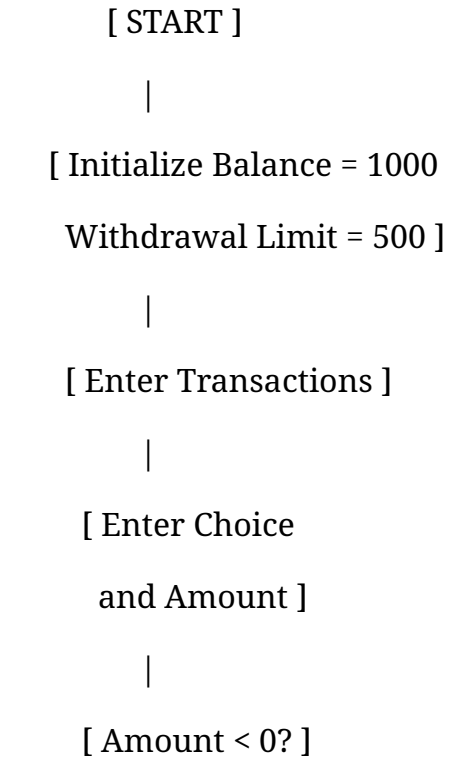
Algorithm

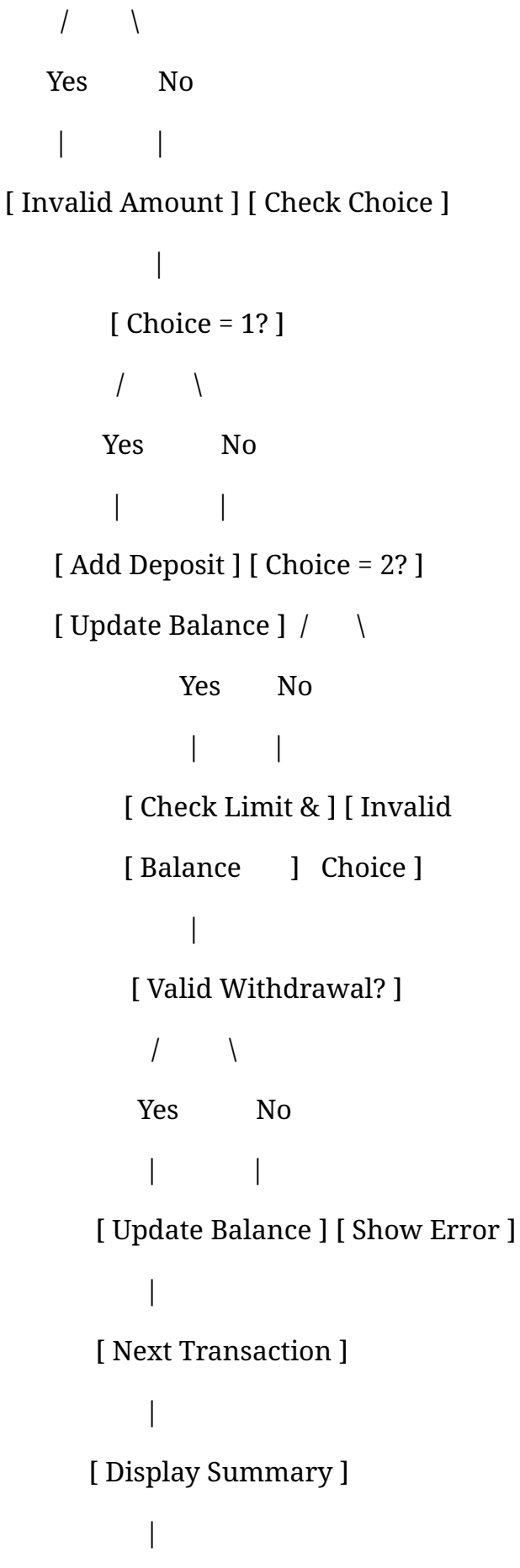
1. Start.
2. Initialize balance = 1000.
3. Set withdrawal_limit = 500.
4. Initialize deposit = 0 and withdrawals = 0.
5. Read the number of transactions n.
6. Repeat the following steps for each transaction:

- Display Deposit and Withdraw options.
- Read the user's choice and transaction amount.
- If the amount is negative, display an invalid transaction message.
- If the choice is 1, add the amount to the balance and total deposits.
- If the choice is 2, check the withdrawal limit and available balance.
- If the withdrawal is valid, subtract the amount from the balance and update total withdrawals.
- Otherwise, display the appropriate error message.
- If the choice is invalid, display an invalid choice message.

7. Display total deposits.
8. Display total withdrawals.
9. Display the final account balance.
10. Stop.

Flowchart




/ \
Yes No
| |
[Invalid Amount] [Check Choice]
|
[Choice = 1?]
/ \
Yes No
| |
[Add Deposit] [Choice = 2?]
[Update Balance] / \
Yes No
| |
[Check Limit &] [Invalid
[Balance] Choice]
|
[Valid Withdrawal?]
/ \
Yes No
| |
[Update Balance] [Show Error]
|
[Next Transaction]
|
[Display Summary]
|

[STOP]

Pseudocode

START

balance = 1000

withdrawal_limit = 500

deposit = 0

withdrawals = 0

Read n

FOR each transaction from 1 to n

  Display Deposit and Withdraw options

  Read choice and amount

  IF amount < 0

    Display "Invalid transaction"

  ELSE IF choice == 1

    balance = balance + amount

    deposit = deposit + amount

    Display "Deposit successful"

  ELSE IF choice == 2

    IF amount > withdrawal_limit

```
            Display "Withdrawal limit exceeded"

        ELSE IF amount > balance
            Display "Insufficient balance"

        ELSE
            balance = balance - amount
            withdrawals = withdrawals + amount
            Display "Withdrawal successful"

    ELSE
        Display "Invalid choice"

    ENDIF

ENDFOR

Display total deposits
Display total withdrawals
Display final balance

STOP
```

Python Source Code

```
balance = 1000
withdrawal_limit = 500
```

```
deposit = 0
withdrawals = 0

n = int(input("Enter the number of transactions: "))

for i in range(n):
    print("\n1. Deposit")
    print("2. Withdraw")

    choice = int(input("Enter your choice: "))
    amount = float(input("Enter your amount: "))

    if amount < 0:
        print("Invalid transaction!")

    elif choice == 1:
        balance = balance + amount
        deposit = deposit + amount
        print("Deposit successful!")

    elif choice == 2:
        if amount > withdrawal_limit:
            print("Withdrawal limit exceeded!")

        elif amount > balance:
            print("Insufficient balance!")
```

```
        else:
            balance = balance - amount
            withdrawals = withdrawals + amount
            print("Withdrawal successful!")

    else:
        print("Invalid choice!")

print("\n==== Transaction Summary ====")
print("Total deposits:", deposit)
print("Total withdrawals:", withdrawals)
print("Final balance:", balance)
```

Test Cases

Test Case 1: Successful deposit and withdrawal

Input:

```
Enter the number of transactions: 2

1. Deposit
2. Withdraw
Enter your choice: 1
Enter your amount: 500

1. Deposit
2. Withdraw
Enter your choice: 2
```

Enter your amount: 300

Output:

Deposit successful!

Withdrawal successful!

==== Transaction Summary ====

Total deposits: 500.0

Total withdrawals: 300.0

Final balance: 1200.0

Test Case 2: Withdrawal limit exceeded

Input:

Enter the number of transactions: 1

1. Deposit

2. Withdraw

Enter your choice: 2

Enter your amount: 600

Output:

Withdrawal limit exceeded!

==== Transaction Summary ====

Total deposits: 0

Total withdrawals: 0

Final balance: 1000

Conclusion

This program manages basic banking transactions using Python. It demonstrates loops, conditional statements, arithmetic operations, and input validation. It also checks withdrawal limits and available balance to prevent invalid withdrawals and displays a summary of all successful transactions.

Case Study: Bus Route Capacity and Passenger Demand Management

Problem Statement:

Bus Route Capacity and Passenger Demand Management: Calculate the number of buses required for multiple routes based on passenger demand and bus capacity. Determine the available seating capacity and identify the number of passengers who cannot be accommodated.

Objective

- Accept the number of routes.
- Enter passenger demand and bus capacity for each route.
- Calculate the number of buses required.
- Calculate the total available passenger capacity.
- Identify the number of unserved passengers.
- Display the results for each route.

Assumptions

- Passenger demand and bus capacity are entered by the user.
- Each bus has the same capacity for its respective route.
- The number of buses required is rounded up to a whole number.
- Passenger demand must be zero or greater, and bus capacity must be greater than zero.
- All buses are assumed to operate at their full seating capacity.

Algorithm

1. Start.
2. Read the number of routes.
3. Repeat for each route.
4. Display the route number.
5. Read passenger demand and bus capacity.
6. Calculate the number of buses using the formula:

buses = (passengers + capacity - 1) // capacity

7. Calculate the available capacity:

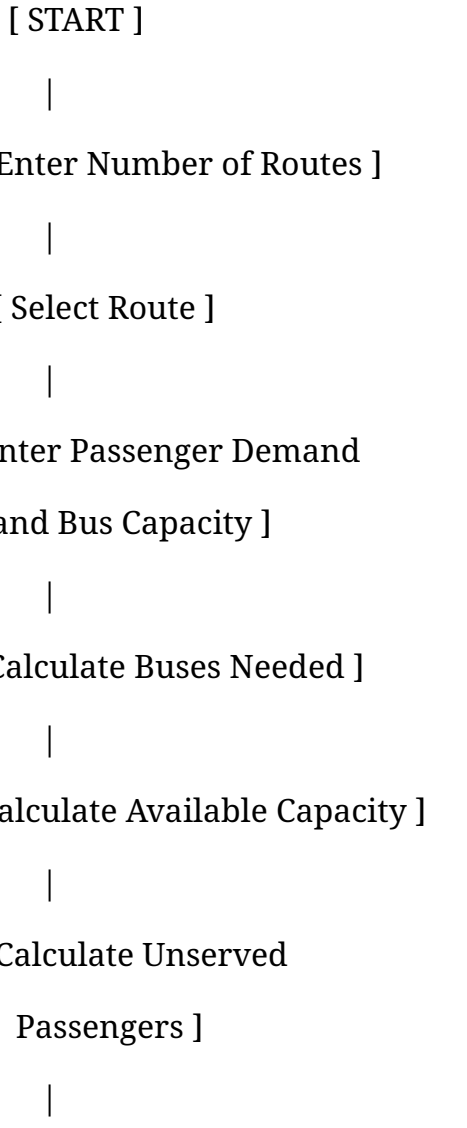
available_capacity = buses × capacity

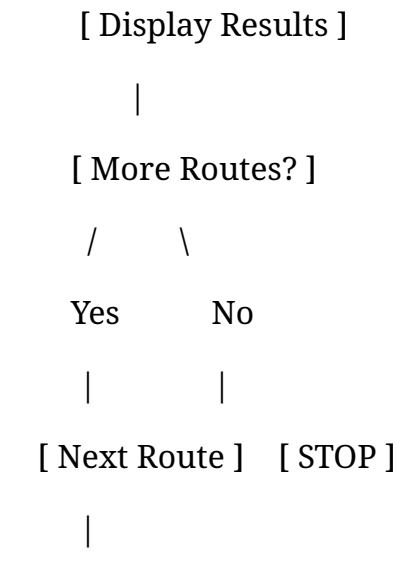
8. Calculate unserved passengers:

unserved = max(0, passengers - available_capacity)

9. Display passenger demand, bus capacity, buses required, and unserved passengers.
10. Repeat for all routes.
11. Stop.

Flowchart





Pseudocode

START

Read routes

FOR each route

Read passengers

Read capacity

buses = (passengers + capacity - 1) // capacity

available_capacity = buses * capacity

unserved = MAX(0, passengers - available_capacity)

Display passengers

Display capacity

Display buses

Display unserved

ENDFOR

STOP

Python Source Code

```
routes = int(input("Enter the number of routes: "))

for i in range(routes):
    print("\nRoute", i + 1)

    passengers = int(input("Enter the passenger demand: "))
    capacity = int(input("Enter the capacity of bus: "))

    buses = (passengers + capacity - 1) // capacity

    available_capacity = buses * capacity
    unserved = max(0, passengers - available_capacity)

    print("Passengers:", passengers)
    print("Bus capacity:", capacity)
    print("Buses needed:", buses)
    print("Unserved passengers:", unserved)
```

Note: In the given code, unserved is always zero for valid positive bus capacity because the program calculates enough buses to accommodate every passenger. The unused seats are available_capacity - passengers.

Test Cases

Test Case 1: Passenger demand is within capacity

Input:

Enter the number of routes: 1

Route 1

Enter the passenger demand: 80

Enter the capacity of bus: 40

Output:

Passengers: 80

Bus capacity: 40

Buses needed: 2

Unserved passengers: 0

Test Case 2: Passenger demand exceeds one bus capacity

Input:

Enter the number of routes: 1

Route 1

Enter the passenger demand: 95

Enter the capacity of bus: 40

Output:

Passengers: 95

Bus capacity: 40

Buses needed: 3

Unserved passengers: 0

Conclusion

This program helps determine the number of buses needed to meet passenger demand on different routes. It demonstrates the use of loops, arithmetic operators, integer division, and the max() function in Python. It can help with basic bus scheduling and capacity planning.

Case Study: Traffic Violation Penalty Calculator

Problem Statement:

Traffic Violation Penalty Calculator: Calculate traffic fines for multiple violations based on vehicle type, offence type, and repeat-offence status. Determine the total penalty and identify serious violations that may lead to driver disqualification.

Objective

- Accept the number of traffic violations.
- Enter the vehicle type, offence type, and repeat-offence status.
- Calculate fines according to predefined rules.
- Double the fine for repeat offences.
- Calculate the total penalty.
- Identify serious offences that may require checking driver disqualification rules.

Assumptions

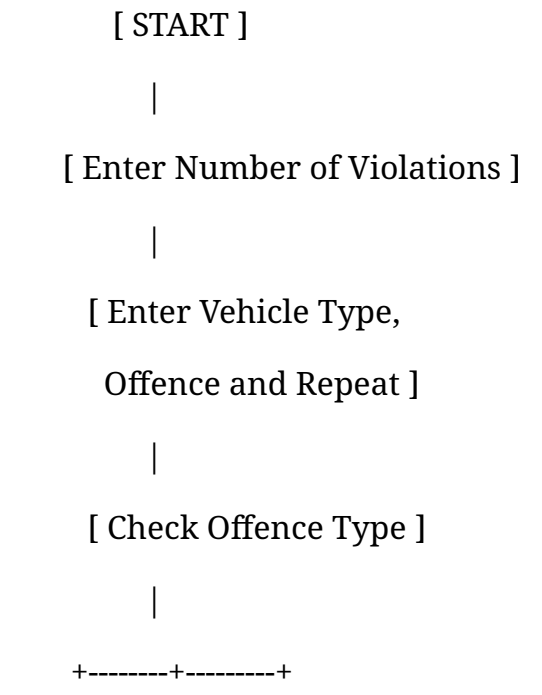
- Vehicle types are bike and car.
- Offence types include speeding, no helmet, and drunk driving.
- The fine for speeding is ₹1,000.
- The fine for riding a bike without a helmet is ₹500.
- The fine for drunk driving is ₹10,000.
- Repeat offences double the applicable fine.
- Drunk driving is treated as a serious offence by this program.
- These are example values for an educational program, not a statement of current legal penalties.

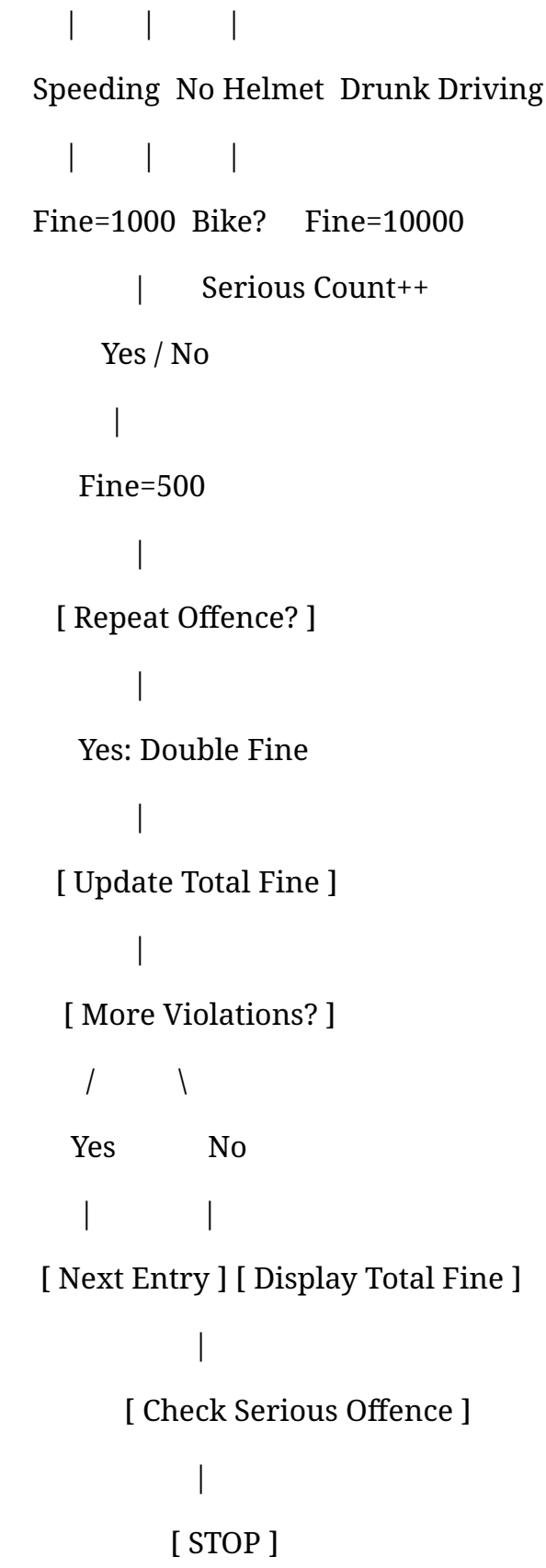
Algorithm

1. Start.
2. Read the number of violations.

3. Initialize total_fine = 0 and serious_count = 0.
4. Repeat for each violation.
5. Read vehicle type, offence type, and repeat-offence status.
6. If the offence is speeding, set the fine to ₹1,000.
7. If the offence is no helmet and the vehicle is a bike, set the fine to ₹500.
8. If the offence is drunk driving, set the fine to ₹10,000 and increase serious_count.
9. If the offence or vehicle combination is invalid, display an error message.
10. If the offence is repeated, double the applicable fine.
11. Add the fine to total_fine.
12. Display the fine for each violation.
13. Display the total penalty.
14. If serious_count is greater than zero, display a possible disqualification warning.
15. Stop.

Flowchart





Pseudocode

START

```
Read n

total_fine = 0
serious_count = 0

FOR each violation

    Read vehicle
    Read offence
    Read repeat status

    IF offence == "speeding"
        fine = 1000

    ELSE IF offence == "no helmet" AND vehicle == "bike"
        fine = 500

    ELSE IF offence == "drunk driving"
        fine = 10000
        serious_count = serious_count + 1

    ELSE
        fine = 0
        Display "Invalid offence or vehicle combination"
```

```
    ENDIF

    IF repeat == "yes" AND fine > 0
        fine = fine * 2
    ENDIF

    total_fine = total_fine + fine

    Display fine

ENDFOR

Display total_fine

IF serious_count > 0
    Display "Possible driver disqualification"
ELSE
    Display "No disqualification condition detected"
ENDIF

STOP
```

Python Source Code

```
# Traffic Penalty Calculator

n = int(input("Enter the number of violations: "))
```

```
total_fine = 0
serious_count = 0

for i in range(n):
    print("\nViolation", i + 1)

    vehicle = input("Enter vehicle type (bike/car): ").lower()
    offence = input(
        "Enter offence (speeding/no helmet/drunk driving): "
    ).lower()
    repeat = input("Is it a repeat offence? (yes/no): ").lower()

    if offence == "speeding":
        fine = 1000

    elif offence == "no helmet" and vehicle == "bike":
        fine = 500

    elif offence == "drunk driving":
        fine = 10000
        serious_count += 1

    else:
        fine = 0
        print("Invalid offence or vehicle combination!")
```

```
    if repeat == "yes" and fine > 0:
        fine = fine * 2

    total_fine = total_fine + fine

    print("Fine for this violation: Rs.", fine)

print("\nTotal Penalty: Rs.", total_fine)

if serious_count > 0:
    print("Warning: Possible driver disqualification.")
    print("Check applicable law.")
else:
    print("No disqualification condition detected.")
```

Note: I corrected the variable name veichel to vehicle and made the offence names consistent. Enter no helmet and drunk driving with spaces, exactly as shown in the prompts.

Test Cases

Test Case 1: Speeding offence

Input:

Enter the number of violations: 1

Violation 1

Enter vehicle type (bike/car): car

Enter offence (speeding/no helmet/drunk driving): speeding

Is it a repeat offence? (yes/no): no

Output:

Fine for this violation: Rs. 1000

Total Penalty: Rs. 1000

No disqualification condition detected.

Test Case 2: Repeat drunk-driving offence

Input:

Enter the number of violations: 1

Violation 1

Enter vehicle type (bike/car): car

Enter offence (speeding/no helmet/drunk driving): drunk driving

Is it a repeat offence? (yes/no): yes

Output:

Fine for this violation: Rs. 20000

Total Penalty: Rs. 20000

Warning: Possible driver disqualification.

Check applicable law.

Conclusion

This program calculates traffic penalties using Python conditional statements, loops, and arithmetic operations. It considers vehicle types, offence categories, and repeat offences to calculate individual fines and the total penalty. It also identifies serious offences that may require further checking under applicable traffic laws.

Case Study: Campus Event Financial Planning

Problem Statement:

Campus Event Financial Planning: Calculate the total cost and income for a campus event based on venue, equipment, catering, transport, certificates, participant registrations, and sponsorships. Determine the financial shortfall and the minimum number of participants required to break even.

Objective

- Accept event expenses and expected participants.
- Calculate fixed costs and per-person costs.
- Calculate the total event cost.
- Calculate registration income and sponsorship income.
- Determine the financial shortfall.
- Calculate the minimum participants required to break even.

Assumptions

- Venue, equipment, and transport costs are fixed costs.
- Catering and certificate costs depend on the number of participants.
- Each participant pays the same registration fee.
- Sponsorship is treated as event income.
- Break-even is possible only when the registration fee is greater than the cost per participant.
- All amounts are entered in Indian Rupees (₹).

Algorithm

1. Start.
2. Read venue, equipment, catering, transport, and certificate costs.
3. Read expected participants, sponsorship amount, and registration fee.
4. Calculate fixed cost as venue + equipment + transport.
5. Calculate per-person cost as catering + certificate.

6. Calculate total event cost using fixed cost and participant expenses.
7. Calculate registration income by multiplying the registration fee by participants.
8. Calculate total income by adding registration income and sponsorship.
9. Calculate shortfall as the difference between total cost and total income, or zero if there is no shortfall.
10. If the registration fee is greater than the per-person cost, calculate the minimum participants required to break even.
11. Otherwise, display that break-even is not possible with the given registration fee.
12. Display the financial summary.
13. Stop.

Flowchart

[START]

|

[Enter Event Costs]

|

[Enter Participants,

Sponsorship and Fee]

|

[Calculate Fixed Cost]

|

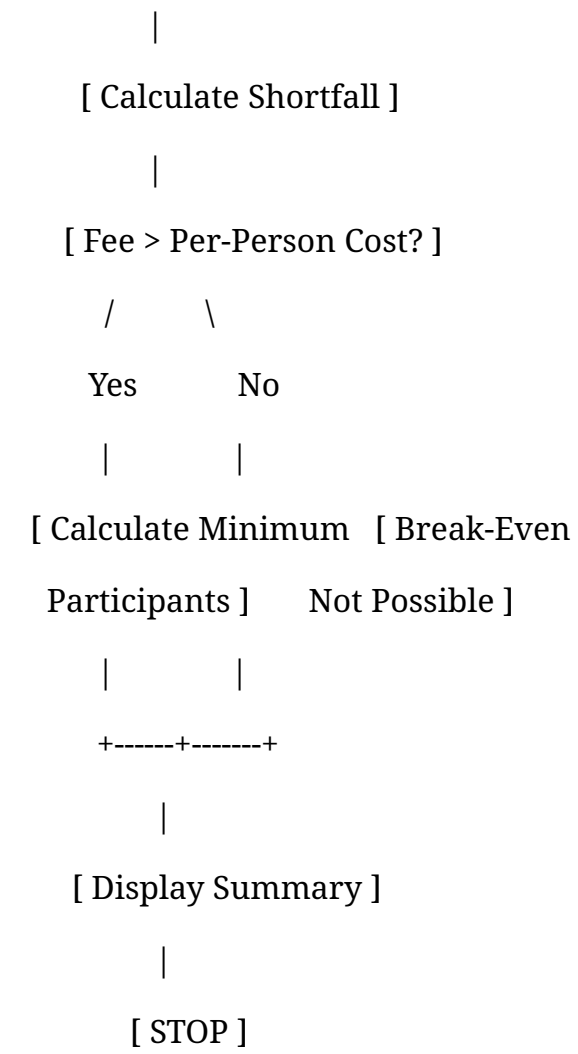
[Calculate Per-Person Cost]

|

[Calculate Total Cost]

|

[Calculate Total Income]



Pseudocode

START

Read venue, equipment, catering, transport, certificate

Read participants, sponsorship, contribution

fixed_cost = venue + equipment + transport

person_cost = catering + certificate

total_cost = fixed_cost + person_cost * participants

```
registration_money = contribution * participants
total_income = registration_money + sponsorship

shortfall = MAX(0, total_cost - total_income)

IF contribution > person_cost
   required = (fixed_cost - sponsorship) /
            (contribution - person_cost)

   IF required < 0
      required = 0
   ENDIF

   Round required up to the next integer
   Display minimum participants
ELSE
   Display "Break-even not possible"
ENDIF

Display total_cost
Display total_income
Display shortfall

STOP
```

Python Source Code

```
# Campus Event Financial Planning

venue = float(input("Enter venue cost: Rs. "))
equipment = float(input("Enter equipment cost: Rs. "))
catering = float(input("Enter catering cost per person: Rs. "))
transport = float(input("Enter transport cost: Rs. "))
certificate = float(input("Enter certificate cost per person: Rs. "))

participants = int(input("Enter expected participants: "))
sponsorship = float(input("Enter sponsorship amount: Rs. "))
contribution = float(input("Enter registration fee per person: Rs. "))

fixed_cost = venue + equipment + transport
person_cost = catering + certificate

total_cost = fixed_cost + (person_cost * participants)

registration_money = contribution * participants
total_income = registration_money + sponsorship

shortfall = max(0, total_cost - total_income)

if contribution > person_cost:
    required = (fixed_cost - sponsorship) / (contribution - person_cost)

    if required < 0:
```

```
        required = 0

    required = int(required) + (required % 1 > 0)
else:
    required = 0

print("\n--- EVENT FINANCIAL SUMMARY ---")
print("Fixed Cost: Rs.", fixed_cost)
print("Total Event Cost: Rs.", total_cost)
print("Registration Income: Rs.", registration_money)
print("Sponsorship: Rs.", sponsorship)
print("Total Income: Rs.", total_income)
print("Sponsorship Shortfall: Rs.", shortfall)

if contribution > person_cost:
    print("Minimum Participants to Break Even:", required)
else:
    print("Break-even not possible with this registration fee.")
```

Test Cases

Test Case 1: Event with sufficient income

Input:

Enter venue cost: Rs. 10000

Enter equipment cost: Rs. 5000

Enter catering cost per person: Rs. 200

Enter transport cost: Rs. 3000

Enter certificate cost per person: Rs. 50

Enter expected participants: 100

Enter sponsorship amount: Rs. 10000

Enter registration fee per person: Rs. 500

Output:

--- EVENT FINANCIAL SUMMARY ---

Fixed Cost: Rs. 18000.0

Total Event Cost: Rs. 43000.0

Registration Income: Rs. 50000.0

Sponsorship: Rs. 10000.0

Total Income: Rs. 60000.0

Sponsorship Shortfall: Rs. 0

Minimum Participants to Break Even: 23

Test Case 2: Break-even not possible

Input:

Enter venue cost: Rs. 10000

Enter equipment cost: Rs. 5000

Enter catering cost per person: Rs. 300

Enter transport cost: Rs. 2000

Enter certificate cost per person: Rs. 50

Enter expected participants: 50

Enter sponsorship amount: Rs. 5000

Enter registration fee per person: Rs. 300

Output:

--- EVENT FINANCIAL SUMMARY ---

Fixed Cost: Rs. 17000.0

Total Event Cost: Rs. 34500.0

Registration Income: Rs. 15000.0

Sponsorship: Rs. 5000.0

Total Income: Rs. 20000.0

Sponsorship Shortfall: Rs. 14500.0

Break-even not possible with this registration fee.

Conclusion

This program helps organize campus event finances by calculating expenses, income, and financial shortfalls. It demonstrates Python arithmetic operations, conditional statements, and the max() function. The break-even calculation helps event organizers estimate the minimum number of participants needed to cover fixed costs and per-person expenses.

Case Study: Employee Payroll and Deduction Analysis

Problem Statement:

Employee Payroll and Deduction Analysis: Calculate an employee's gross salary based on basic salary, allowances, overtime, and incentives. Determine tax deductions using progressive tax slabs, calculate Provident Fund (PF) deductions, and display the final net salary.

Objective

- Accept employee details and salary components.
- Calculate gross salary.
- Calculate income tax using progressive tax slabs.
- Calculate PF deductions based on basic salary.
- Calculate total deductions.
- Determine and display the employee's net salary.

Assumptions

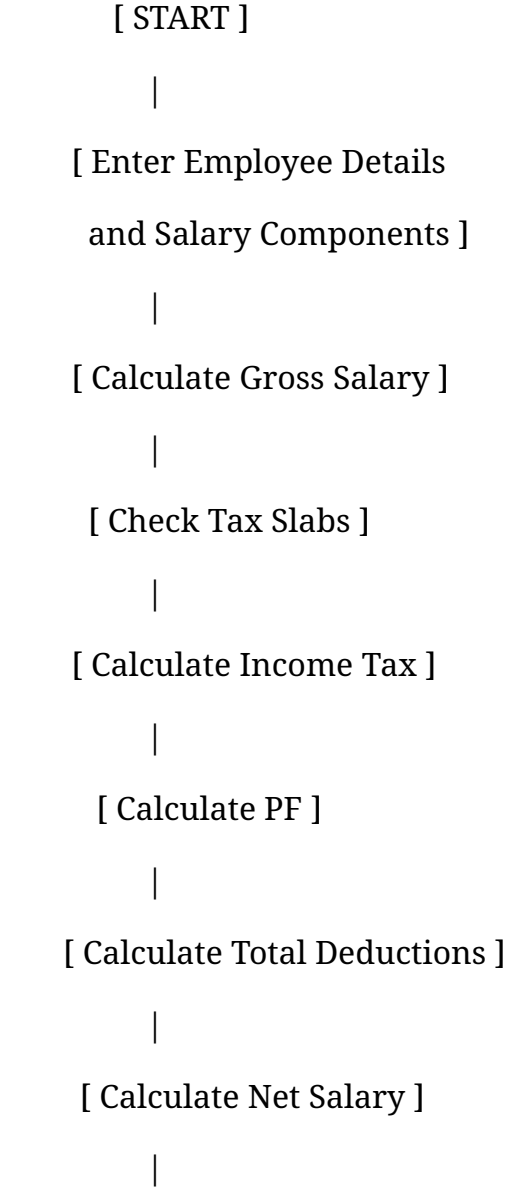
- Basic salary, allowances, overtime, and incentives are entered in Indian Rupees (₹).
- Gross salary is the sum of all four salary components.
- Tax rates are 5% up to ₹15,000, 10% on the next ₹15,000, and 15% on the amount above ₹30,000.
- PF deduction is 12% of basic salary.
- Tax rates are illustrative values used for this programming case study, not official tax advice.

Algorithm

1. Start.
2. Read employee name.
3. Read basic salary, allowances, overtime, and incentives.
4. Calculate gross salary by adding all salary components.
5. If gross salary is less than or equal to ₹15,000, calculate tax at 5%.

6. If gross salary is between ₹15,000 and ₹30,000, calculate 5% tax on the first ₹15,000 and 10% on the remaining amount.
7. If gross salary exceeds ₹30,000, calculate tax using all three slabs.
8. Calculate PF as 12% of basic salary.
9. Calculate total deductions by adding tax and PF.
10. Calculate net salary by subtracting total deductions from gross salary.
11. Display the salary statement.
12. Stop.

Flowchart



[Display Salary Statement]

|

[STOP]

Pseudocode

START

Read name

Read basic, allowances, overtime, incentives

gross = basic + allowances + overtime + incentives

IF gross <= 15000

tax = gross * 0.05

ELSE IF gross <= 30000

tax = (15000 * 0.05) + ((gross - 15000) * 0.10)

ELSE

tax = (15000 * 0.05) + (15000 * 0.10)

+ ((gross - 30000) * 0.15)

ENDIF

pf = basic * 0.12

total_deductions = tax + pf

net_salary = gross - total_deductions

Display gross

Display tax

Display pf

Display total_deductions

Display net_salary

STOP

Python Source Code

```
# Employee Payroll and Deduction Analysis

name = input("Enter employee name: ")
basic = float(input("Enter basic salary: Rs. "))
allowances = float(input("Enter allowances: Rs. "))
overtime = float(input("Enter overtime amount: Rs. "))
incentives = float(input("Enter incentives: Rs. "))

gross = basic + allowances + overtime + incentives

if gross <= 15000:
    tax = gross * 0.05

elif gross <= 30000:
    tax = (15000 * 0.05) + ((gross - 15000) * 0.10)
```

```
else:
    tax = (15000 * 0.05) + (15000 * 0.10) + ((gross - 30000) * 0.15)

pf = basic * 0.12

total_deductions = tax + pf
net_salary = gross - total_deductions

print("\n===== SALARY STATEMENT =====")
print("Employee Name:", name)
print("Basic Salary: Rs.", basic)
print("Allowances: Rs.", allowances)
print("Overtime: Rs.", overtime)
print("Incentives: Rs.", incentives)
print("Gross Salary: Rs.", round(gross, 2))
print("Tax Deduction: Rs.", round(tax, 2))
print("PF Deduction: Rs.", round(pf, 2))
print("Total Deductions: Rs.", round(total_deductions, 2))
print("Net Salary: Rs.", round(net_salary, 2))
```

Test Cases

Test Case 1: Employee with gross salary below ₹15,000

Input:

Enter employee name: Ravi

Enter basic salary: Rs. 10000

Enter allowances: Rs. 2000

Enter overtime amount: Rs. 1000

Enter incentives: Rs. 500

Output:

===== SALARY STATEMENT =====

Employee Name: Ravi

Basic Salary: Rs. 10000.0

Allowances: Rs. 2000.0

Overtime: Rs. 1000.0

Incentives: Rs. 500.0

Gross Salary: Rs. 13500.0

Tax Deduction: Rs. 675.0

PF Deduction: Rs. 1200.0

Total Deductions: Rs. 1875.0

Net Salary: Rs. 11625.0

Test Case 2: Employee with gross salary above ₹30,000

Input:

Enter employee name: Suresh

Enter basic salary: Rs. 25000

Enter allowances: Rs. 5000

Enter overtime amount: Rs. 2000

Enter incentives: Rs. 3000

Output:

===== SALARY STATEMENT =====

Employee Name: Suresh

Basic Salary: Rs. 25000.0

Allowances: Rs. 5000.0

Overtime: Rs. 2000.0

Incentives: Rs. 3000.0

Gross Salary: Rs. 35000.0

Tax Deduction: Rs. 3750.0

PF Deduction: Rs. 3000.0

Total Deductions: Rs. 6750.0

Net Salary: Rs. 28250.0

Conclusion

This program calculates employee salaries and deductions using Python. It demonstrates input handling, arithmetic operations, conditional statements, and progressive tax calculations. It helps determine gross salary, tax, PF, total deductions, and net salary in a clear salary statement.

Case Study: Campus Festival Registration and Financial Audit

Problem Statement:

Campus Festival Registration and Financial Audit: Manage participant registrations by detecting duplicate names, calculate the total registration fees and sponsorship income, compare the total income with festival expenses, and determine whether the festival results in a profit or loss.

Objective

- Accept the number of participants and their names.
- Detect duplicate registrations.
- Calculate total registration fees from valid registrations.
- Accept sponsorship and total expenses.
- Calculate the final financial balance.
- Identify whether the festival makes a profit or incurs a loss.

Assumptions

- Participant names are entered by the user.
- Duplicate names are not registered again.
- Registration fees are collected only from valid participants.
- Sponsorship and expenses are entered as integer amounts in rupees.
- A zero or positive balance indicates no loss; a negative balance indicates a loss.

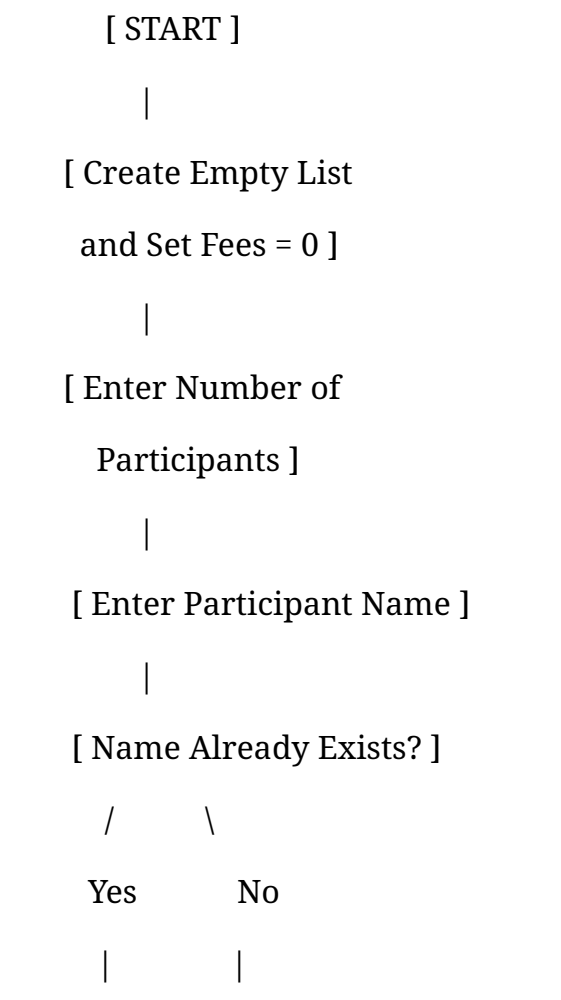
Algorithm

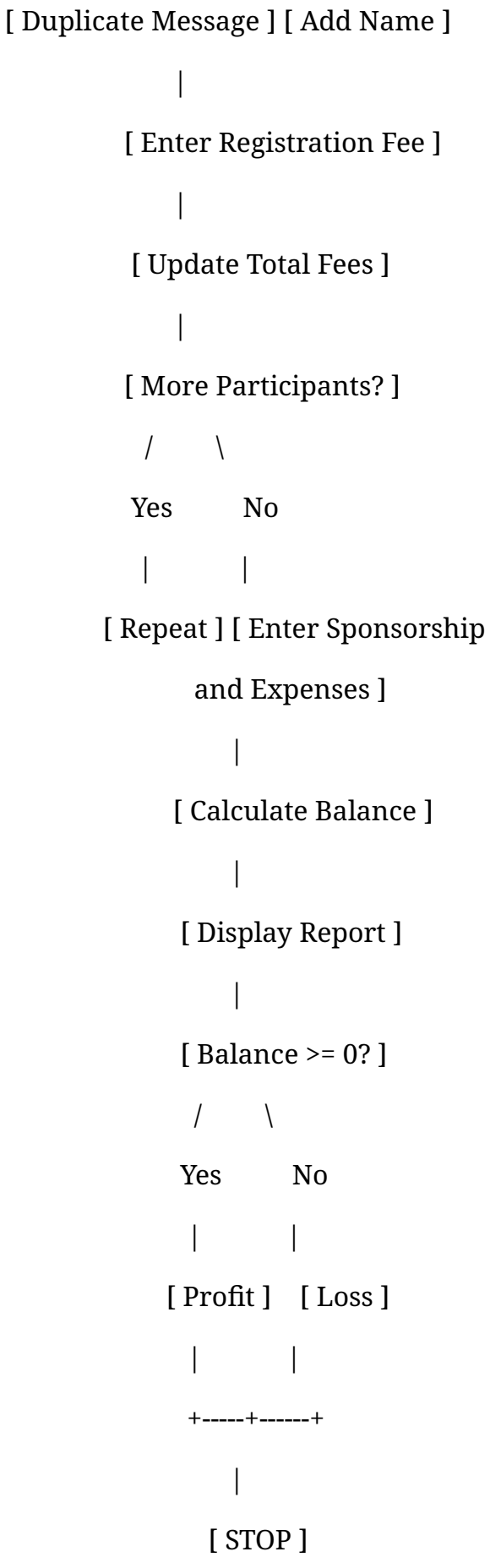
1. Start.
2. Create an empty list called participants.
3. Initialize fees = 0.
4. Read the number of participants.
5. Repeat for each participant:
    - Read the participant's name.

- Check whether the name already exists in the list.
- If it exists, display "Duplicate registration!".
- Otherwise, add the name to the list, read the registration fee, and add it to the total fees.

6. Read the sponsorship amount.
7. Read the total expenses.
8. Calculate the final balance as registration fees + sponsorship − expenses.
9. Display the number of valid participants and the financial report.
10. If the balance is greater than or equal to zero, display Profit; otherwise, display Loss.
11. Stop.

Flowchart




[Duplicate Message] [Add Name]
[Enter Registration Fee]
[Update Total Fees]
[More Participants?]
Yes
No
[Repeat]
[Enter Sponsorship and Expenses]
[Calculate Balance]
[Display Report]
[Balance >= 0?]
Yes
No
[Profit]
[Loss]
[STOP]

Pseudocode

```
START

participants = empty list
fees = 0

Read n

FOR each participant

    Read name

    IF name exists in participants
        Display "Duplicate registration!"
    ELSE
        Add name to participants
        Read fee
        fees = fees + fee
    ENDIF

ENDFOR

Read sponsorship
Read expenses

balance = fees + sponsorship - expenses
```

Display number of valid participants

Display fees, sponsorship, expenses, balance

IF balance >= 0

  Display "Festival Result: Profit"

ELSE

  Display "Festival Result: Loss"

ENDIF

STOP

Python Source Code

```
# Campus Festival Registration and Financial Audit

participants = []
fees = 0

n = int(input("Enter number of participants: "))

for i in range(n):
    name = input("Enter participant name: ")

    if name in participants:
        print("Duplicate registration!")
    else:
```

```
        participants.append(name)
        fee = int(input("Enter registration fee: "))
        fees = fees + fee

sponsorship = int(input("Enter sponsorship amount: "))
expenses = int(input("Enter total expenses: "))

balance = fees + sponsorship - expenses

print("\n--- Festival Report ---")
print("Valid Participants:", len(participants))
print("Registration Fees:", fees)
print("Sponsorship:", sponsorship)
print("Expenses:", expenses)
print("Final Balance:", balance)

if balance >= 0:
    print("Festival Result: Profit")
else:
    print("Festival Result: Loss")
```

Test Cases

Test Case 1: Festival makes a profit

Input:

Enter number of participants: 3

Enter participant name: Ravi

Enter registration fee: 500

Enter participant name: Sita

Enter registration fee: 500

Enter participant name: Kumar

Enter registration fee: 500

Enter sponsorship amount: 2000

Enter total expenses: 2500

Output:

--- Festival Report ---

Valid Participants: 3

Registration Fees: 1500

Sponsorship: 2000

Expenses: 2500

Final Balance: 1000

Festival Result: Profit

Test Case 2: Duplicate registration

Input:

Enter number of participants: 3

Enter participant name: Ravi

Enter registration fee: 500

Enter participant name: Sita

Enter registration fee: 500

Enter participant name: Ravi

Duplicate registration!

Enter sponsorship amount: 500

Enter total expenses: 2000

Output:

Duplicate registration!

--- Festival Report ---

Valid Participants: 2

Registration Fees: 1000

Sponsorship: 500

Expenses: 2000

Final Balance: -500

Festival Result: Loss

Conclusion

This program manages campus festival registrations and finances using Python lists, loops, conditional statements, and arithmetic operations. It prevents duplicate registrations, calculates total income and expenses, and determines whether the festival makes a profit or incurs a loss.

Case Study: Water Distribution Fairness Analysis

Problem Statement:

Water Distribution Fairness Analysis: Monitor water demand and supply across multiple areas over several days. Calculate the total demand, total supply, water shortage, and supply percentage for each area, and identify the most underserved area.

Objective

- Define the areas receiving water.
- Accept the number of days for analysis.
- Record daily water demand and supply for each area.
- Calculate total demand and supply for each area.
- Determine water shortages and supply percentages.
- Identify the area with the highest water shortage.

Assumptions

- There are three areas: Area A, Area B, and Area C.
- Demand and supply are entered as non-negative integer values in a consistent unit, such as liters.
- Daily demand and supply are accumulated over the specified number of days.
- Shortage is calculated when demand exceeds supply.
- Supply percentage represents the percentage of demand that was supplied.
- The most underserved area is identified by the highest total shortage.

Algorithm

1. Start.
2. Create a list of three areas: Area A, Area B, and Area C.
3. Initialize demand and supply values to zero for all areas.
4. Read the number of days.

5. Repeat for each day:
    - Repeat for each area.
    - Read the water demand and supply.
    - Add the values to the area's total demand and supply.
6. Create an empty list called shortages.
7. For each area:
    - Calculate shortage as the maximum of zero and demand minus supply.
    - Store the shortage in the list.
    - Calculate supply percentage if demand is greater than zero; otherwise, set it to zero.
    - Display total demand, total supply, shortage, and supply percentage.
8. Find the area with the highest shortage.
9. Display the most underserved area and its shortage.
10. Stop.

Flowchart

[START]

|

[Initialize Areas,

Demand and Supply]

|

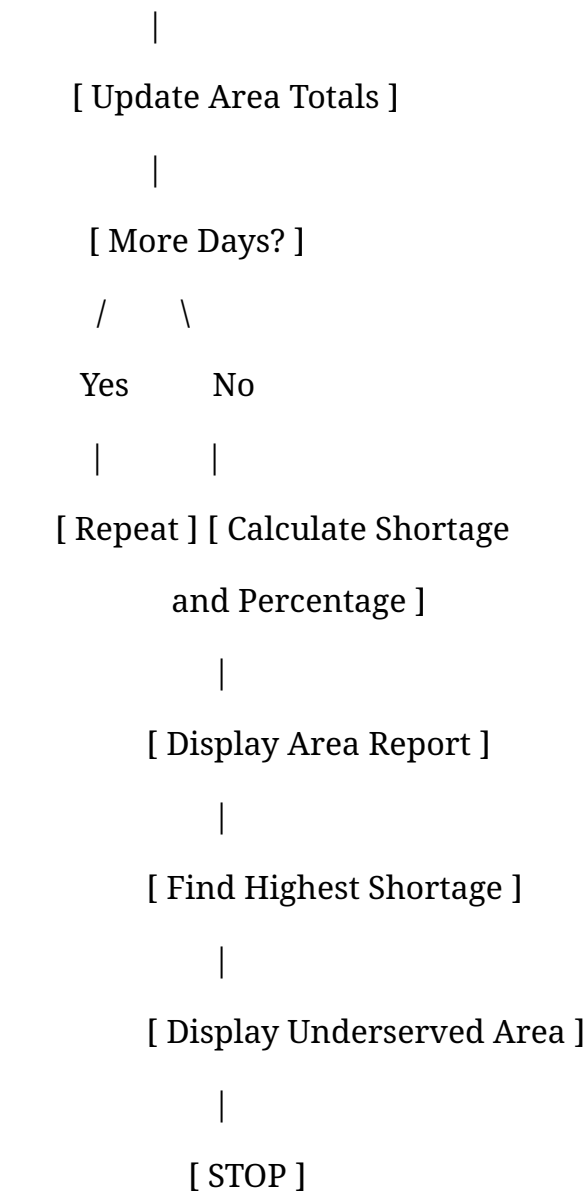
[Enter Number

of Days]

|

[Enter Daily

Demand and Supply]



Pseudocode

START

areas = [Area A, Area B, Area C]

Initialize demand and supply to 0

Read days

FOR each day

```
    FOR each area
        Read daily demand
        Read daily supply

        Add demand to total demand
        Add supply to total supply
    ENDFOR
ENDFOR

FOR each area
    shortage = MAX(0, demand - supply)
    Store shortage

    IF demand > 0
        percentage = supply / demand * 100
    ELSE
        percentage = 0
    ENDIF

    Display demand, supply, shortage, percentage
ENDFOR

Find area with highest shortage

Display most underserved area
Display highest shortage
```

STOP

Python Source Code

```
# Water Distribution Fairness Analysis

areas = ["Area A", "Area B", "Area C"]
demand = [0, 0, 0]
supply = [0, 0, 0]

days = int(input("Enter number of days: "))

for day in range(days):
    print("\nDay", day + 1)

    for i in range(3):
        d = int(input("Enter demand for " + areas[i] + ": "))
        s = int(input("Enter supply for " + areas[i] + ": "))

        demand[i] += d
        supply[i] += s

print("\n--- Water Distribution Report ---")

shortages = []
```

```
for i in range(3):
    shortage = max(0, demand[i] - supply[i])
    shortages.append(shortage)

    if demand[i] > 0:
        percentage = supply[i] / demand[i] * 100
    else:
        percentage = 0

    print("\nArea:", areas[i])
    print("Total Demand:", demand[i])
    print("Total Supply:", supply[i])
    print("Shortage:", shortage)
    print("Supply Percentage:", round(percentage, 2), "%")

worst = shortages.index(max(shortages))

print("\nMost Underserved Area:", areas[worst])
print("Highest Shortage:", shortages[worst])
```

Test Cases

Test Case 1: Water shortage in multiple areas

Input:

Enter number of days: 1

Day 1

Enter demand for Area A: 1000

Enter supply for Area A: 800

Enter demand for Area B: 1200

Enter supply for Area B: 900

Enter demand for Area C: 800

Enter supply for Area C: 700

Output:

--- Water Distribution Report ---

Area: Area A

Total Demand: 1000

Total Supply: 800

Shortage: 200

Supply Percentage: 80.0 %

Area: Area B

Total Demand: 1200

Total Supply: 900

Shortage: 300

Supply Percentage: 75.0 %

Area: Area C

Total Demand: 800

Total Supply: 700

Shortage: 100

Supply Percentage: 87.5 %

Most Underserved Area: Area B

Highest Shortage: 300

Test Case 2: No water shortage

Input:

Enter number of days: 1

Day 1

Enter demand for Area A: 500

Enter supply for Area A: 600

Enter demand for Area B: 400

Enter supply for Area B: 400

Enter demand for Area C: 300

Enter supply for Area C: 350

Output:

--- Water Distribution Report ---

Area: Area A

Total Demand: 500

Total Supply: 600

Shortage: 0

Supply Percentage: 120.0 %

Area: Area B

Total Demand: 400

Total Supply: 400

Shortage: 0

Supply Percentage: 100.0 %

Area: Area C

Total Demand: 300

Total Supply: 350

Shortage: 0

Supply Percentage: 116.67 %

Most Underserved Area: Area A

Highest Shortage: 0

Note: When all areas have zero shortage, the program still displays Area A because it is the first area in the list. Also, a supply percentage above 100% means the area received more water than its recorded demand.

Conclusion

This program analyzes water distribution across multiple areas over several days using Python lists, nested loops, conditional statements, and arithmetic operations. It calculates total demand, supply, shortages, and supply percentages, helping identify areas that may require additional water resources.

Case Study: Mobile Plan Cost Optimization System

1. Problem Statement

A telecommunications company offers different mobile plans with different prices, data limits, and validity periods. A customer enters their budget, required data, and required validity. The program compares three mobile plans and identifies the cheapest plan that satisfies all the customer's requirements.

2. Objective

- To compare three mobile plans.
- To check the required data and validity.
- To ensure the plan price is within the customer's budget.
- To find the cheapest suitable plan.
- To display a message if no suitable plan is available.

3. Assumptions

- The customer enters a valid budget, data requirement, and validity period.
- Exactly three mobile plans are compared.
- Plan prices and data amounts are entered as integers.
- A plan is suitable only if it satisfies all three conditions.
- The cheapest suitable plan is selected.

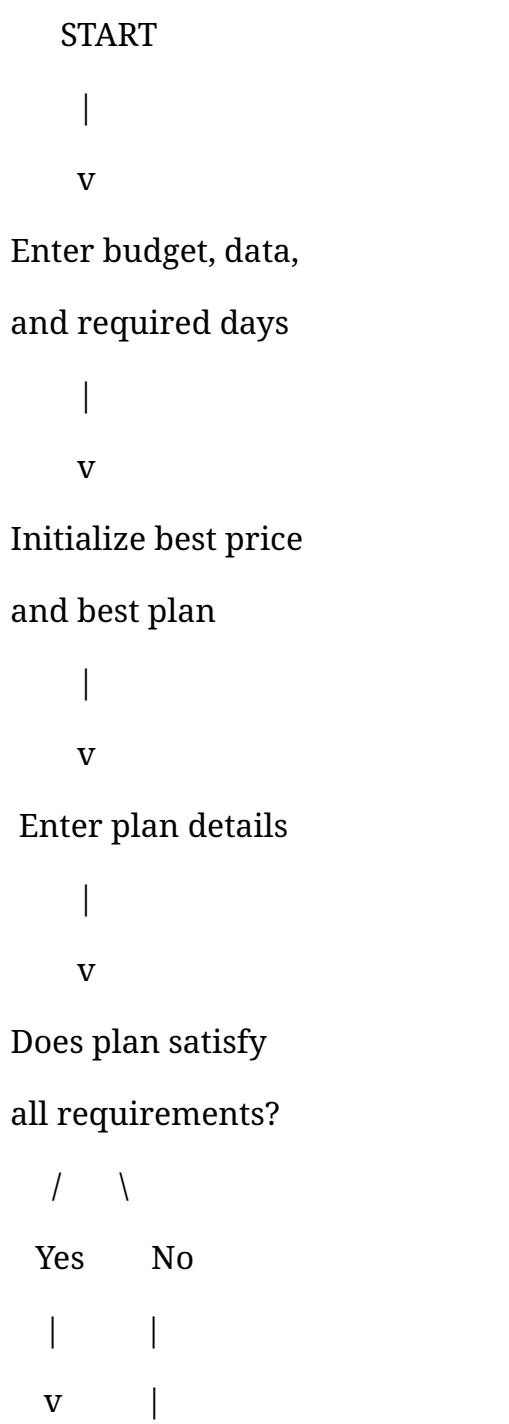
4. Algorithm

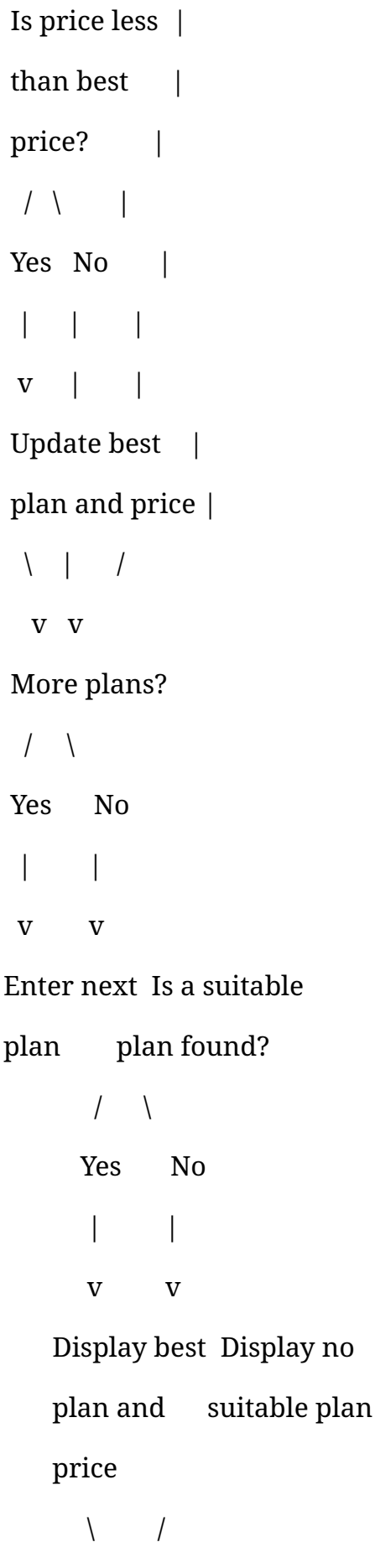
1. Start the program.
2. Read the customer's budget, required data, and required validity.
3. Set the best price to a large value and the best plan to "No plan".
4. Repeat the following steps for three plans:
    - Read the plan price, data, and validity.
    - Check whether the plan meets the customer's requirements.

- If it is suitable and cheaper than the current best plan, update the best plan.

5. If a suitable plan is found, display its name and price.
6. Otherwise, display "No suitable plan found".
7. Stop the program.

5. Flowchart




Is price less
than best
price?
Yes
No
Update best
plan and price
More plans?
Yes
No
Enter next
plan
Is a suitable
plan found?
Yes
No
Display best
plan and
price
Display no
suitable plan

v

END

6. Pseudocode

START

INPUT budget

INPUT required_data

INPUT required_days

best_price = 99999

best_plan = "No plan"

FOR each of 3 plans

INPUT price

INPUT plan_data

INPUT validity

IF plan_data >= required_data AND

validity >= required_days AND

price <= budget THEN

IF price < best_price THEN

best_price = price

best_plan = current plan

END IF

```
    END IF
END FOR

IF best_plan is not "No plan" THEN
    DISPLAY best_plan
    DISPLAY best_price
ELSE
    DISPLAY "No suitable plan found"
END IF

STOP
```

7. Python Source Code

```
budget = int(input("Enter your budget: "))
data = int(input("Enter required data in GB: "))
days = int(input("Enter required validity in days: "))

best_price = 99999
best_plan = "No plan"

for i in range(3):
    print("\nEnter details for Plan", i + 1)

    price = int(input("Enter plan price: "))
    plan_data = int(input("Enter data in GB: "))
    validity = int(input("Enter validity in days: "))
```

```
    if plan_data >= data and validity >= days and price <= budget:
        if price < best_price:
            best_price = price
            best_plan = "Plan " + str(i + 1)

if best_plan != "No plan":
    print("\nBest plan:", best_plan)
    print("Price:", best_price)
else:
    print("\nNo suitable plan found")
```

8. Test Cases

Test Case 1: Suitable plan found

Input:

Enter your budget: 300

Enter required data in GB: 2

Enter required validity in days: 28

Enter details for Plan 1

Enter plan price: 250

Enter data in GB: 2

Enter validity in days: 28

Enter details for Plan 2

Enter plan price: 199

Enter data in GB: 1

Enter validity in days: 28

Enter details for Plan 3

Enter plan price: 299

Enter data in GB: 3

Enter validity in days: 56

Output:

Best plan: Plan 1

Price: 250

Plan 1 and Plan 3 meet the requirements, but Plan 1 costs less.

Test Case 2: No suitable plan found

Input:

Enter your budget: 150

Enter required data in GB: 3

Enter required validity in days: 56

Plan 1: Price = 200, Data = 3 GB, Validity = 56 days

Plan 2: Price = 100, Data = 1 GB, Validity = 28 days

Plan 3: Price = 180, Data = 4 GB, Validity = 56 days

Output:

No suitable plan found

None of the plans satisfies all the requirements within the budget.

9. Conclusion

The Mobile Plan Cost Optimization System compares three mobile plans based on price, data, validity, and budget. It selects the cheapest plan that meets the customer's needs. This program helps customers choose an economical mobile plan using simple Python loops and conditional statements.

Case Study: Food Waste Monitoring System

Problem Statement:

Food Waste Monitoring System: Monitor the number of meals prepared, served, and left over across several days. Calculate the food wastage percentage for each day and identify days when wastage exceeds the acceptable limit.

Objective

- Accept the number of days for monitoring.
- Record meals prepared, meals served, and leftover meals each day.
- Validate the entered data.
- Calculate the food wastage percentage.
- Compare the wastage percentage with the acceptable limit.
- Display a warning when food wastage is too high.

Assumptions

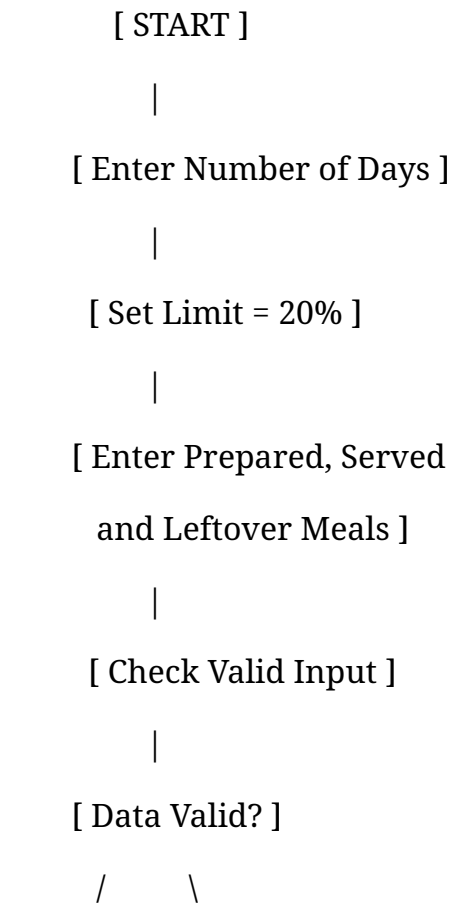
- The number of days is entered by the user.
- Meals prepared must be greater than zero.
- Meals served and leftover meals must be non-negative.
- The sum of meals served and leftover meals must not exceed meals prepared.
- The acceptable food wastage limit is 20%.
- Wastage percentage is calculated using leftover meals divided by prepared meals, multiplied by 100.

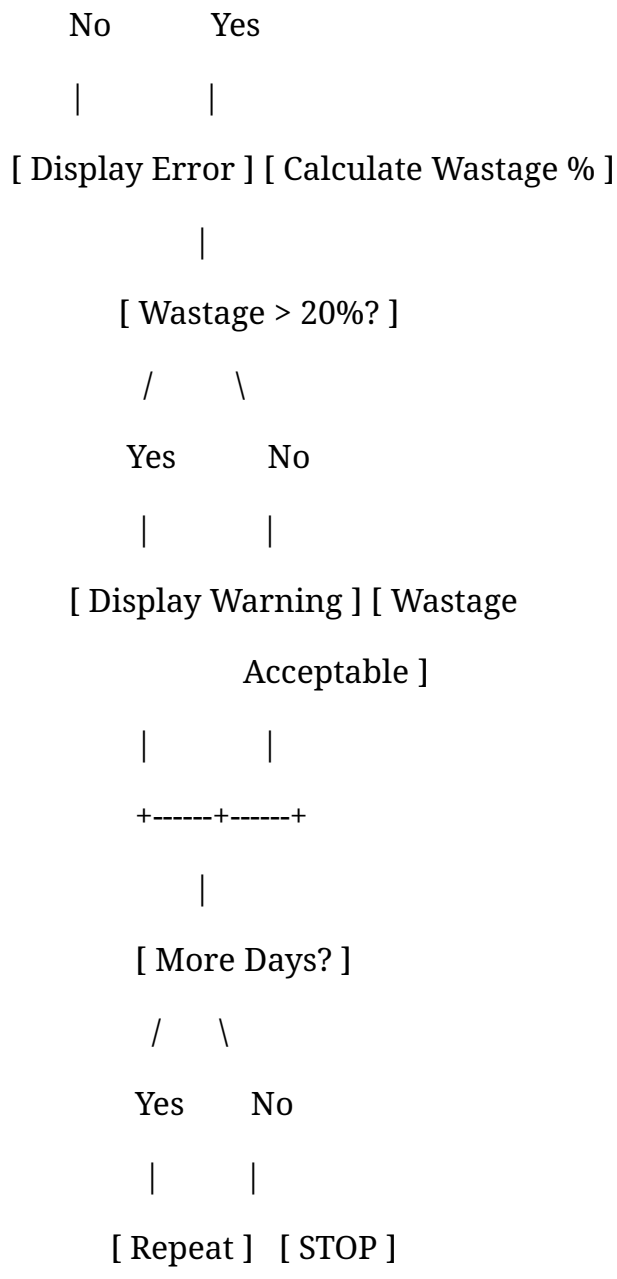
Algorithm

1. Start.
2. Read the number of days.
3. Set the acceptable wastage limit to 20%.
4. Repeat for each day.

5. Read meals prepared, meals served, and leftover meals.
6. Check whether the input values are valid.
7. If prepared meals are zero or negative, or served or leftover meals are negative, display an invalid input message.
8. If served meals plus leftover meals exceed prepared meals, display an invalid data message.
9. Otherwise, calculate the wastage percentage.
10. Display the wastage percentage.
11. If the percentage exceeds 20%, display a warning.
12. Otherwise, display that food wastage is acceptable.
13. Repeat for all days.
14. Stop.

Flowchart





Pseudocode

START

Read days

limit = 20

FOR each day

```
    Read prepared
    Read served
    Read leftovers

    IF prepared <= 0 OR served < 0 OR leftovers < 0
        Display "Invalid input"

    ELSE IF served + leftovers > prepared
        Display "Invalid data"

    ELSE
        percentage = (leftovers / prepared) * 100

        Display percentage

        IF percentage > limit
            Display "Food wastage is too high"
        ELSE
            Display "Food wastage is acceptable"
        ENDIF
    ENDIF

ENDFOR

STOP
```

Python Source Code

```
# Food Waste Monitoring System

days = int(input("Enter number of days: "))

limit = 20

for i in range(days):
    print("\nDay", i + 1)

    prepared = int(input("Enter meals prepared: "))
    served = int(input("Enter meals served: "))
    leftovers = int(input("Enter leftover meals: "))

    if prepared <= 0 or served < 0 or leftovers < 0:
        print("Invalid input")

    elif served + leftovers > prepared:
        print("Invalid data: meals exceed prepared meals")

    else:
        percentage = (leftovers / prepared) * 100

        print("Wastage percentage:", round(percentage, 2), "%")

        if percentage > limit:
```

```
        print("Warning: Food wastage is too high!")
    else:
        print("Food wastage is acceptable.")
```

Test Cases

Test Case 1: High food wastage

Input:

Enter number of days: 1

Day 1

Enter meals prepared: 100

Enter meals served: 60

Enter leftover meals: 40

Output:

Wastage percentage: 40.0 %

Warning: Food wastage is too high!

Test Case 2: Acceptable food wastage

Input:

Enter number of days: 1

Day 1

Enter meals prepared: 100

Enter meals served: 85

Enter leftover meals: 15

Output:

Wastage percentage: 15.0 %

Food wastage is acceptable.

Conclusion

This program monitors food waste using Python loops, conditional statements, arithmetic operations, and input validation. It calculates daily wastage percentages and warns when wastage exceeds 20%, helping kitchens, hostels, and cafeterias reduce food waste and improve meal planning.